

3 RAID 5 磁盘阵列

3.1 背景

尽管单个磁盘的故障率相对较低，但当多个磁盘在同一系统中协同工作时，整体故障风险显著增加。例如，假设单个磁盘的年故障率为 1%，对于一个包含 10 个磁盘的存储系统，年内至少一个磁盘发生故障的概率就会上升到约 9.56%。这种情况下，RAID 技术通过将数据和校验信息分布于多个磁盘上，增强了系统的容错和数据恢复能力，以应对由单一或多个磁盘故障带来的潜在数据丢失风险。这一机制对于维护企业级存储系统的稳定性和可靠性至关重要。

本次考试中，你需要以多个文件读写模拟多个磁盘的读写，实现一个 RAID 5 磁盘阵列。

3.2 奇偶校验

在 RAID 5 中，奇偶校验用于数据的恢复。奇偶校验块是通过将同一组中的数据块逐位进行异或 (XOR) 运算得到的。例如，假设我们有三个数据块，每个块由四位构成：数据块 A_1 是 0001，数据块 A_2 是 0101，数据块 A_3 是 0011。它们的奇偶校验块 A_p 计算如下：

- 对第一位进行异或： $0 \oplus 0 \oplus 0 = 0$
- 对第二位进行异或： $0 \oplus 1 \oplus 0 = 1$
- 对第三位进行异或： $0 \oplus 0 \oplus 1 = 1$
- 对第四位进行异或： $1 \oplus 1 \oplus 1 = 1$

因此，奇偶校验块 A_p 为 0111。奇偶校验的一个重要性质是，对于每一位，数据块和奇偶校验块的 1 的总数都是偶数。在上文的例子中，第一位有 0 个 1，第二位有 2 个 1，第三位有 2 个 1，第四位有 4 个 1。这意味着即使一个块丢失，也可以通过其余数据块和奇偶校验块重建丢失的数据。例如，如果数据块 A_1 丢失，我们可以通过 $(A_2 \oplus A_3 \oplus A_p)$ 计算出数据块 A_1 的值。在 C++ 中，可以使用 ^ 运算符进行异或运算，例如表达式 $1 \wedge 5 \wedge 3$ 的结果是 7。

3.3 RAID 5 简要介绍

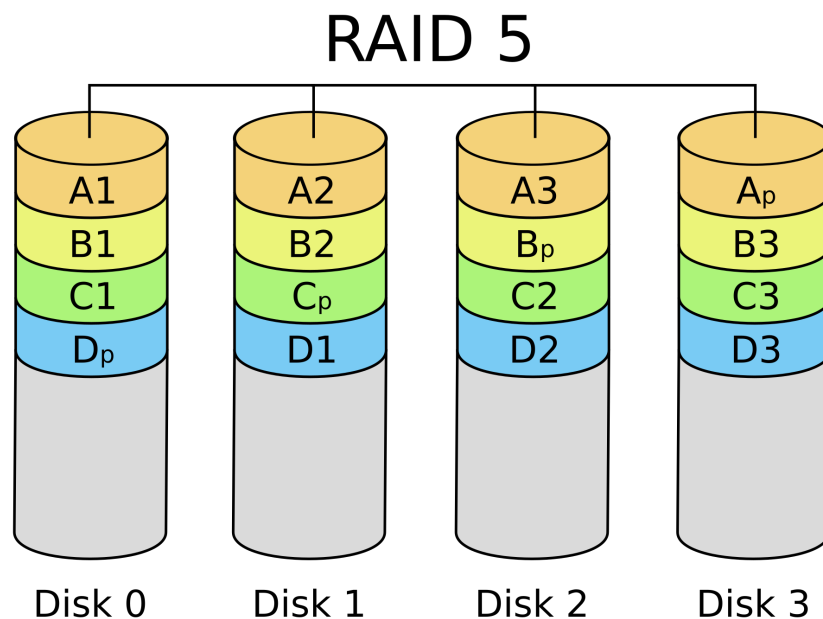


图 1: RAID5 示意图

如图所示, RAID 5 是一种将数据块分布式存储的磁盘阵列技术。它需要至少 3 个磁盘, 通过将数据和校验信息分布在所有磁盘上来提供容错能力。

一个具有 n 个磁盘的 RAID 5 阵列有以下特点:

1. 数据被分成多个块, 分布存储在不同磁盘上。例如, 图中的 **A1**、**A2**、**A3**、**B1**、**B2**、**B3**、**C1**、**C2**、**C3** 分别代表第 0、1、2、3、4、5、6、7、8 个块
2. 每 $n - 1$ 个块分为一组。例如, 图中的 **A1**、**A2**、**A3** 为第 0 组, **B1**、**B2**、**B3** 为第 1 组, **C1**、**C2**、**C3** 为第 2 组
3. 每组数据都有一个对应的校验块, 用于数据恢复。校验块中的数据为组内其它数据块的异或和 (奇偶校验)
4. 校验块被轮流存储在不同的磁盘上, 使磁盘的写入次数更均匀
5. 可以在一个磁盘故障的情况下, 通过其他磁盘的数据和校验信息恢复数据
6. 磁盘空间的具体组织方式为 (编号均从 0 开始):
 - 每个磁盘从开头开始的第 i 个块存储第 i 组的数据块或校验块
 - 第 i 组的校验块存储在第 $n - 1 - i \bmod n$ 个磁盘上
 - 每组除了校验块外的 $n - 1$ 个数据块从左到右依次存储在剩余的磁盘上

当 RAID 5 阵列中有一个磁盘故障时, 系统会进入“降级模式”(Degraded Mode) 运行。在降级模式下:

1. 系统仍然可以正常读写数据, 但性能会下降

2. 对于读取操作，如果请求的数据块恰好在故障的磁盘上，系统会通过其他磁盘的数据和校验信息实时计算出所需数据
3. 对于写入操作，如果数据块或校验块恰好在故障的磁盘上，系统会通过其他磁盘的数据和校验信息实时计算出所需数据并写入
4. 在更换新磁盘之前，系统处于无容错状态，此时如果再有磁盘故障，将导致数据无法恢复

3.4 要求

在本题中，我们将使用多个文件来模拟多个磁盘，每个文件代表一个磁盘。数据将被分块存储在这些文件中，并计算相应的校验信息。当其中一个“磁盘”（文件）故障时，系统应能够通过其他文件中的数据和校验信息恢复出完整数据。同时，系统应尽量减少对磁盘的读写次数。

为了简化问题，本题中磁盘只会在系统“关机”期间发生故障或更换。在系统“开机”时会告知发生的事件类型和对应的磁盘编号。在系统“关机”时，需要将所有打开的文件关闭。另外，在同一时间，最多只有一个磁盘处于故障状态。

具体而言，需要支持的操作如下：

```

1  #include "fstream.h"
2  #include <vector>
3
4  // 磁盘事件类型：正常、故障、更换
5  enum class EventType {
6      NORMAL, // 正常：所有磁盘工作正常
7      FAILED, // 故障：指定磁盘发生故障（文件被删除）
8      REPLACED // 更换：指定磁盘被更换（文件被清空）
9  };
10
11
12 class RAID5Controller {
13 private:
14     std::vector<sjtu::fstream*> drives_; // 磁盘文件对应的 fstream 对象
15     int blocks_per_drive_; // 每个磁盘的块数
16     int block_size_; // 每个块的大小
17     int num_disks_; // 磁盘数
18     // 允许添加其它成员变量和成员函数
19 public:
20     RAID5Controller(std::vector<sjtu::fstream*> drives, int blocks_per_drive,
21                     int block_size = 4096) {
22         // 输入“磁盘”（文件）对应的 fstream* 对象。
23         // 文件已经打开，且大小为 block_size * blocks_per_drive
24         // 文件初始数据为全 0
25     }
26
27     /**
28      * @brief 启动 RAID5 系统
29      * @param event_type_ 磁盘事件类型
30      * @param drive_id 发生事件的磁盘编号（如果是 NORMAL 则忽略）
31      *
32      * 如果是 FAILED，对应的磁盘文件会被删除。此时不可再对该文件进行读写，系统进入降级模式。
33      * 如果是 REPLACED，对应的磁盘文件中的数据会全部清零。此时应立即利用其它磁盘的数据和校验信息恢复数据。
34      * 如果是 NORMAL，所有磁盘正常工作
35      * 注：磁盘被替换之前不一定发生故障。
36      */
37     void Start(EventType event_type_, int drive_id) {
38         // TODO
39     }
40

```

```
41 void Shutdown() {  
42     // 关闭所有打开的文件，以防未定义行为发生。  
43 }  
44  
45 void ReadBlock(int block_id, char *result) {  
46     // 读取第 block_id 个块的内容，写入 result 中  
47 }  
48  
49 void WriteBlock(int block_id, const char *data) {  
50     // 将 data 中的内容写入第 block_id 个块中  
51 }  
52  
53 int Capacity() {  
54     // 返回磁盘阵列能写入的块的数量  
55     return (num_disks_ - 1) * blocks_per_drive_;  
56 }  
57 };
```

3.5 注意

1. 本题中所有 `block_id` 从 0 开始计数。
2. `RAID5Controller` 类被构造时初始处于关机状态，调用 `Start` 方法后进入开机状态，调用 `Shutdown` 方法后进入关机状态。评测数据保证 `Start` 方法只会在关机状态下调用，`ReadBlock`、`WriteBlock` 和 `Shutdown` 方法只会在开机状态下调用。评测时同一个 `RAID5Controller` 对象会调用多次 `Start` 和 `Shutdown` 方法。
3. 第一次调用 `Start` 方法时，所有磁盘均处于正常状态。
4. 代码框架中的 `sjtu::fstream` 的所有接口均与 `std::fstream` 一致，但在 OJ 测试中会自动记录每个文件的读写次数和读写总量。本地下发文件中包含了 `fstream.h` 头文件，你可以直接使用其中的 `sjtu::fstream` 类。该类仅供调试使用，与 OJ 测试时所用的 `fstream` 类不同。
5. 评测程序会释放传入的 `drives` 数组中的指针，你只需要在 `Shutdown` 方法中关闭文件，不需要释放指针。
6. 禁止将文件内容存储在内存中，所有数据均需实时读写。违者将被助教手动判为 0 分。
7. 禁止在文件的规定区域之外 (`blocks_per_drive_ * block_size_` 字节之后) 进行读写，也禁止读写其它文件。违者将被助教手动判为 0 分。

3.6 读写次数限制

一个性能良好的 Raid 5 磁盘阵列应该做到：

- 执行每个 `ReadBlock` 操作时，系统会从磁盘中读取平均不超过 2 个块的数据。
- 执行每个 `WriteBlock` 操作时，系统会向磁盘中写入平均不超过 2 个块的数据，并向磁盘中写入平均不超过 2 个块的数据。
- 替换磁盘时，系统会读取 `num_disks_ - 1` 块磁盘中的全部数据，并向替换之后磁盘中写入 `blocks_per_drive_` 个块数据。
- 每块磁盘的读写次数应尽量均匀。

为了检验你的程序是否满足上述要求，我们会在 Subtask 5 中对你的程序对磁盘读写次数进行限制。具体地，我们在程序结束前检查每个磁盘文件的总读写次数。

在 Subtask 5 中，只有磁盘被替换，没有磁盘故障，评测程序调用 `ReadBlock` 和 `WriteBlock` 时的 `block_id` 为均匀随机分布，替换磁盘时的 `drive_id` 也为均匀随机分布。

设系统中有 N 个磁盘，每个磁盘有 M 个数据块，数据块大小为 B ，评测程序调用 `ReadBlock` R 次，调用 `WriteBlock` W 次，替换磁盘 P 次，则：

- 系统对**每个**磁盘文件的读入次数不得超过为 $\frac{(R+2W+(N-1)MP)}{N} \times 1.1$ ，读入总量不得超过 $\frac{(R+2W+(N-1)MP) \times B}{N} \times 1.1$
- 系统对**每个**磁盘文件的写入次数不得超过为 $\frac{2W+MP}{N} \times 1.1$ ，写入总量不得超过 $\frac{(2W+MP) \times B}{N} \times 1.1$

3.7 数据范围

本题采用 Subtask 给分制。

- Subtask 1 (20 %): 没有磁盘故障和替换
- Subtask 2 (20 %): 只有磁盘被替换，没有磁盘故障
- Subtask 3 (20 %): 既有磁盘故障，也有磁盘替换。在磁盘故障时，会有读操作，但没有写操作。
- Subtask 4 (20 %): 既有磁盘故障，也有磁盘替换。在磁盘故障时，会有读和写操作。
- Subtask 5 (20 %): 只有磁盘被替换，没有磁盘故障。此子任务会对读写次数进行限制。

对于 Subtask 3 和 4，保证若上次 `Start` 时有磁盘故障，下一次 `Start` 时一定会替换故障的磁盘。但是若上次 `Start` 时没有磁盘故障，下一次 `Start` 时也可能替换任意磁盘。也就是说，任意时刻只有不超过一个磁盘处于故障状态。

在 Subtask 5 中，评测程序调用 `ReadBlock` 和 `WriteBlock` 时的 `block_id` 为均匀随机分布，替换磁盘时的 `drive_id` 也为均匀随机分布。

对于所有数据，保证 $1 \leq \text{block_size} \leq 4096$ ， $1 \leq \text{blocks_per_drive} \leq 100$ ， $3 \leq \text{num_disks} \leq 10$ 。